CSS Positioning, Layout, and Design

Introduction: Mastering Web Layouts with CSS

Building a visually appealing and well-organized website requires more than just styling text and colors. It demands a deep understanding of how to **position** elements on the page, control their **stacking order**, and create **responsive layouts** that adapt to different screen sizes.

CSS provides powerful tools for controlling the placement and arrangement of elements. The `position` property allows you to precisely place elements anywhere on the page. The `float` property enables text to wrap around images and creates multi-column layouts. The `z-index` property controls which elements appear on top of others. Combined with **image galleries** and **pseudo-classes** for interactive effects, these tools allow developers to create professional, polished websites.

This reading material examines how to use CSS positioning, floats, and z-index to control layout, create image galleries with CSS, and apply pseudo-classes and pseudo-elements for interactive and decorative effects. By the end of this lesson, learners will be able to design advanced web layouts and create visually engaging web pages.

Part 1: The CSS Position Property

Introduction to Positioning

The `position` property in CSS determines how an element is placed in the document. It allows you to control the layout of elements with precision, moving them from their normal position or fixing them in place.

Position Values

Value	Description
`static`	Default positioning; elements flow normally in the document
`relative`	Positioned relative to its normal position
`absolute`	Positioned relative to the nearest positioned ancestor
`fixed`	Positioned relative to the viewport (stays fixed on scroll)
`sticky`	Toggles between relative and fixed based on scroll position

The `static` Position (Default)

All elements have `position: static` by default. They appear in the normal flow of the document.

```
css
.element {
    position: static; /* Default behavior */
}
```

The `relative` Position

An element with `position: relative` is positioned relative to its normal position. Using `top`, `right`, `bottom`, and `left` moves it from where it would normally be.

```
css
.relative-box {
    position: relative;
    top: 20px;   /* Moves down 20px from its normal position */
    left: 30px;  /* Moves right 30px from its normal position */
}
```

Important: Other elements still occupy the original space as if the element never moved.

Example:

```
html
<!DOCTYPE html>
<html>
    <head>
        <title>Relative Positioning</title>
        <style>
            .box {
                width: 200px;
                height: 100px;
                background: #3498db;
                color: white;
                padding: 10px;
                margin: 10px;
            }
            .relative-box {
                position: relative;
                top: 30px;
                left: 50px;
                background: #e74c3c;
            }
        </style>
    </head>
    <body>
        <div class="box">Static Box 1</div>
        <div class="box relative-box">Relative Box (moved)</div>
        <div class="box">Static Box 2</div>
    </body>
</html>
```

The `absolute` Position

An element with `position: absolute` is removed from the normal flow and positioned relative to its nearest positioned ancestor (any element with `position` other than `static`). If no positioned ancestor exists, it positions relative to the `<body>`.

```
css
.absolute-box {
```

```
    position: absolute;
    top: 50px;
    right: 20px;
    background: #2ecc71;
}
```

Key Points:

- Removed from normal flow (other elements shift to fill the gap)
- Positioned using `top`, `right`, `bottom`, `left`
- Relative to the nearest positioned ancestor

Example with a Relative Container:

```
html
<!DOCTYPE html>
<html>
    <head>
        <title>Absolute Positioning</title>
        <style>
            .container {
                position: relative;
                width: 400px;
                height: 300px;
                background: #ecf0f1;
                border: 2px solid #2c3e50;
            }
            .absolute-box {
                position: absolute;
                bottom: 20px;
                right: 20px;
                background: #e74c3c;
                color: white;
                padding: 15px;
                border-radius: 5px;
            }
            .absolute-box2 {
                position: absolute;
                top: 50%;
                left: 50%;
```

```
                transform: translate(-50%, -50%);
                background: #3498db;
                color: white;
                padding: 15px;
                border-radius: 5px;
            }
        </style>
    </head>
    <body>
        <div class="container">
            <div class="absolute-box">Bottom Right</div>
            <div class="absolute-box2">Centered</div>
        </div>
    </body>
</html>
```

The `fixed` Position

An element with `position: fixed` is positioned relative to the viewport (the browser window). It stays in the same position even when the user scrolls.

```
css
.fixed-nav {
    position: fixed;
    top: 0;
    left: 0;
    width: 100%;
    background: #2c3e50;
    color: white;
    padding: 10px;
    z-index: 1000;
}
```

Use Cases:

- Fixed navigation bars
- "Back to top" buttons
- Persistent headers and footers

The `sticky` Position

An element with `position: sticky` behaves like `relative` until it reaches a specified scroll point, then it becomes `fixed`.

```
css
.sticky-header {
    position: sticky;
    top: 0;
    background: #3498db;
    color: white;
    padding: 15px;
}
```

Use Cases:

- Table headers that stay visible while scrolling
- Section headers that stick to the top

Complete Position Example

```
html
<!DOCTYPE html>
<html>
    <head>
        <title>CSS Positioning</title>
        <style>
            body {
                font-family: Arial, sans-serif;
                margin: 0;
                padding: 0;
            }
            .fixed-nav {
                position: fixed;
                top: 0;
                left: 0;
                width: 100%;
                background: #2c3e50;
                color: white;
```

```
    padding: 15px;
    text-align: center;
    z-index: 1000;
}
.content {
    margin-top: 80px;
    padding: 20px;
}
.container {
    position: relative;
    width: 600px;
    height: 400px;
    background: #ecf0f1;
    border: 2px solid #2c3e50;
    margin: 20px auto;
}
.box {
    padding: 15px;
    color: white;
    border-radius: 5px;
}
.box1 {
    position: absolute;
    top: 10px;
    left: 10px;
    background: #e74c3c;
}
.box2 {
    position: absolute;
    top: 10px;
    right: 10px;
    background: #3498db;
}
.box3 {
    position: absolute;
    bottom: 10px;
    left: 10px;
    background: #2ecc71;
}
```

```
            .box4 {
                position: absolute;
                bottom: 10px;
                right: 10px;
                background: #f39c12;
            }
            .sticky-header {
                position: sticky;
                top: 60px;
                background: #9b59b6;
                color: white;
                padding: 10px;
                text-align: center;
            }
        </style>
    </head>
    <body>
        <div class="fixed-nav">Fixed Navigation Bar</div>

        <div class="content">
            <h1>Positioning Examples</h1>

            <div class="container">
                <div class="box box1">Top Left</div>
                <div class="box box2">Top Right</div>
                <div class="box box3">Bottom Left</div>
                <div class="box box4">Bottom Right</div>
            </div>

            <div class="sticky-header">Sticky Header (scroll to see effect)</div>
            <p style="height: 800px;">Scroll down to see the sticky header in action.
</p>
        </div>
    </body>
</html>
```

Part 2: CSS Float and Clear

Introduction to Floats

The `float` property was originally designed to allow text to wrap around images. It later became a common tool for creating multi-column layouts before modern layout systems like Flexbox and Grid.

Float Values

Value	Description
`left`	Element floats to the left; text wraps to the right
`right`	Element floats to the right; text wraps to the left
`none`	Default; element does not float
`inherit`	Inherits the float value from its parent

Using Float

```
css
img {
    float: left;
    margin-right: 15px;
    width: 150px;
}
```

Example: Text Wrapping Around an Image

```
html
<!DOCTYPE html>
<html>
```

```
    <head>
        <title>Float Example</title>
        <style>
            .float-left {
                float: left;
                width: 200px;
                margin: 0 15px 10px 0;
                border: 2px solid #3498db;
            }
            .float-right {
                float: right;
                width: 200px;
                margin: 0 0 10px 15px;
                border: 2px solid #e74c3c;
            }
        </style>
    </head>
    <body>
        <h2>Float Example</h2>
        <div class="float-left">
            <p>This box floats to the left.</p>
        </div>
        <p>This paragraph wraps around the floated box. The text flows alongside the box, filling the available space. This is the original purpose of the float property — to allow text to wrap around images and other elements.</p>
        <div class="float-right">
            <p>This box floats to the right.</p>
        </div>
        <p>This paragraph wraps around the right-floated box. Notice how the text flows around both boxes, filling the space between them.</p>
    </body>
</html>
```

The Clear Property

The `clear` property specifies which side of a floated element an element should not be next to. It is used to prevent elements from wrapping around floats.

Value	Description
`left`	Element moves below left-floated elements
`right`	Element moves below right-floated elements
`both`	Element moves below both left and right floats
`none`	Default; allows both sides

Example:

```
css
.clearfix {
    clear: both;
}
```

```
html
<div class="float-left">Left</div>
<div class="float-right">Right</div>
<div class="clearfix">This appears below both floated elements.</div>
```

The Clearfix Technique

When all children inside a container are floated, the container collapses to zero height. The **clearfix** technique fixes this by adding an invisible element that clears the floats.

```
css
.container::after {
    content: "";
    display: block;
    clear: both;
}
```

Complete Float and Clear Example

```
<!DOCTYPE html>

<html>
```

```
<head>
    <title>Float and Clear</title>
    <style>
        .container {
            border: 2px solid #2c3e50;
            padding: 10px;
            background: #ecf0f1;
        }
        .container::after {
            content: "";
            display: block;
            clear: both;
        }
        .float-left {
            float: left;
            width: 45%;
            background: #3498db;
            color: white;
            padding: 10px;
            margin: 5px;
            box-sizing: border-box;
        }
        .float-right {
            float: right;
            width: 45%;
            background: #e74c3c;
```

```
                color: white;
                padding: 10px;
                margin: 5px;
                box-sizing: border-box;
            }
            .clear-both {
                clear: both;
                background: #2ecc71;
                color: white;
                padding: 10px;
                margin: 5px;
            }
        </style>
    </head>
    <body>
        <div class="container">
            <h2>Float and Clear</h2>
            <div class="float-left">Float Left</div>
            <div class="float-left">Float Left 2</div>
            <div class="float-right">Float Right</div>
            <div class="float-right">Float Right 2</div>
            <div class="clear-both">Clear Both – this appears below all floated elements</div>
        </div>
    </body>
</html>
```

Part 3: CSS Z-Index – Controlling Stacking Order

Introduction to Z-Index

The `z-index` property controls the stacking order of positioned elements. Elements with a higher `z-index` appear in front of elements with a lower `z-index`.

Key Points:

- Only works on **positioned** elements (`relative`, `absolute`, `fixed`, `sticky`)
- Higher value = closer to the user (appears on top)
- Can be negative (appears behind)
- Default value is `auto` (same as `0`)

Syntax:

```
css
.element1 {
    position: absolute;
    z-index: 10;
}
.element2 {
    position: absolute;
    z-index: 5;
}
/* element1 appears on top of element2 */
```

Z-Index Example

```
html
<!DOCTYPE html>
<html>
    <head>
        <title>Z-Index Example</title>
        <style>
            .container {
```

```
            position: relative;
            width: 400px;
            height: 300px;
            background: #ecf0f1;
            margin: 50px auto;
        }
        .box {
            position: absolute;
            padding: 20px;
            color: white;
            font-weight: bold;
            border-radius: 5px;
        }
        .box1 {
            top: 20px;
            left: 20px;
            background: #e74c3c;
            z-index: 1;
        }
        .box2 {
            top: 60px;
            left: 60px;
            background: #3498db;
            z-index: 2;
        }
        .box3 {
            top: 100px;
            left: 100px;
            background: #2ecc71;
            z-index: 3;
        }
        .box4 {
            top: 140px;
            left: 140px;
            background: #f39c12;
            z-index: 0;
        }
    </style>
</head>
```

```
    <body>
        <div class="container">
            <div class="box box1">Box 1 (z-index: 1)</div>
            <div class="box box2">Box 2 (z-index: 2)</div>
            <div class="box box3">Box 3 (z-index: 3)</div>
            <div class="box box4">Box 4 (z-index: 0)</div>
        </div>
    </body>
</html>
```

Part 4: CSS Image Galleries

Introduction to CSS Image Galleries

Image galleries are a common feature on websites, allowing users to view multiple images in an organized layout. CSS can be used to create responsive, visually appealing galleries without JavaScript.

Basic Image Gallery Layout

Using `float` or `flexbox`, images can be arranged in a grid layout.

Float-Based Gallery:

```
html
<!DOCTYPE html>
<html>
    <head>
        <title>Image Gallery</title>
        <style>
            .gallery {
                max-width: 800px;
                margin: 0 auto;
            }
            .gallery::after {
```

```
            content: "";
            display: block;
            clear: both;
        }
        .gallery-item {
            float: left;
            width: 30%;
            margin: 1.5%;
            border: 1px solid #ccc;
            border-radius: 5px;
            text-align: center;
            background: #f9f9f9;
            padding: 5px;
        }
        .gallery-item img {
            width: 100%;
            height: auto;
            border-radius: 5px;
        }
        .gallery-item .desc {
            padding: 10px 0;
            font-size: 14px;
            color: #333;
        }
    </style>
</head>
<body>
    <div class="gallery">
        <div class="gallery-item">
            <img src="image1.jpg" alt="Image 1">
            <div class="desc">Description 1</div>
        </div>
        <div class="gallery-item">
            <img src="image2.jpg" alt="Image 2">
            <div class="desc">Description 2</div>
        </div>
        <div class="gallery-item">
            <img src="image3.jpg" alt="Image 3">
            <div class="desc">Description 3</div>
```

```
            </div>
            <div class="gallery-item">
                <img src="image4.jpg" alt="Image 4">
                <div class="desc">Description 4</div>
            </div>
            <div class="gallery-item">
                <img src="image5.jpg" alt="Image 5">
                <div class="desc">Description 5</div>
            </div>
            <div class="gallery-item">
                <img src="image6.jpg" alt="Image 6">
                <div class="desc">Description 6</div>
            </div>
        </div>
    </body>
</html>
```

Image Gallery with Hover Effects

html

```
<!DOCTYPE html>
<html>
    <head>
        <title>Image Gallery with Hover</title>
        <style>
            .gallery {
                display: flex;
                flex-wrap: wrap;
                justify-content: center;
                gap: 15px;
                max-width: 900px;
                margin: 0 auto;
            }
            .gallery-item {
                position: relative;
                width: 250px;
                overflow: hidden;
                border-radius: 10px;
```

```
            box-shadow: 0 4px 8px rgba(0,0,0,0.2);
            transition: transform 0.3s ease;
        }
        .gallery-item:hover {
            transform: scale(1.05);
        }
        .gallery-item img {
            width: 100%;
            height: 200px;
            object-fit: cover;
            display: block;
        }
        .gallery-item .overlay {
            position: absolute;
            bottom: 0;
            left: 0;
            right: 0;
            background: rgba(0,0,0,0.7);
            color: white;
            padding: 15px;
            transform: translateY(100%);
            transition: transform 0.3s ease;
        }
        .gallery-item:hover .overlay {
            transform: translateY(0);
        }
        .gallery-item .overlay h4 {
            margin: 0 0 5px 0;
        }
        .gallery-item .overlay p {
            margin: 0;
            font-size: 14px;
        }
    </style>
</head>
<body>
    <h1 style="text-align: center;">Image Gallery</h1>
    <div class="gallery">
        <div class="gallery-item">
```

```
            <img src="image1.jpg" alt="Nature">
            <div class="overlay">
                <h4>Nature</h4>
                <p>Beautiful landscape</p>
            </div>
        </div>
        <div class="gallery-item">
            <img src="image2.jpg" alt="City">
            <div class="overlay">
                <h4>City</h4>
                <p>Urban skyline</p>
            </div>
        </div>
        <div class="gallery-item">
            <img src="image3.jpg" alt="Sunset">
            <div class="overlay">
                <h4>Sunset</h4>
                <p>Golden hour</p>
            </div>
        </div>
    </div>
</body>
</html>
```

Part 5: CSS Pseudo-Classes

Introduction to Pseudo-Classes

Pseudo-classes are used to define the special state of an element. They are written with a colon followed by the pseudo-class name (e.g., `:hover`, `:first-child`).

Common Pseudo-Classes

Pseudo-class	Description
`:hover`	Applies when the mouse hovers over an element
`:active`	Applies when an element is being clicked
`:focus`	Applies when an element has focus (e.g., input field)
`:visited`	Applies to visited links
`:link`	Applies to unvisited links
`:first-child`	Selects the first child of a parent
`:last-child`	Selects the last child of a parent
`:nth-child(n)`	Selects the nth child of a parent
`:nth-of-type(n)`	Selects the nth element of its type
`:not(selector)`	Selects all elements that do NOT match the selector

Pseudo-Classes for Links

```
css
/* Unvisited link */
a:link {
    color: #3498db;
}
/* Visited link */
a:visited {
    color: #8e44ad;
}
/* Mouse over link */
a:hover {
    color: #e74c3c;
    text-decoration: underline;
}
/* Clicked link */
a:active {
    color: #2ecc71;
}
```

Pseudo-Classes for Structural Selection

```
css
/* First paragraph in a section */
section p:first-child {
    font-weight: bold;
}
/* Last list item */
li:last-child {
    border-bottom: none;
}
/* Every other list item (even) */
li:nth-child(even) {
    background: #f5f5f5;
}
/* Every third list item */
li:nth-child(3n) {
```

```
    color: red;
}
/* First paragraph after a heading */
h2 + p:first-of-type {
    font-size: 18px;
}
```

Complete Pseudo-Class Example

```
html
<!DOCTYPE html>
<html>
    <head>
        <title>CSS Pseudo-Classes</title>
        <style>
            /* Link states */
            a {
                color: #3498db;
                text-decoration: none;
                font-weight: bold;
            }
            a:hover {
                color: #e74c3c;
                text-decoration: underline;
            }
            a:active {
                color: #2ecc71;
            }

            /* Input focus */
            input:focus {
                background: #f9e79f;
                border: 2px solid #f1c40f;
                outline: none;
            }

            /* Structural pseudo-classes */
            .list-container li:first-child {
```

```
            color: #e74c3c;
            font-weight: bold;
        }
        .list-container li:last-child {
            color: #3498db;
            font-weight: bold;
        }
        .list-container li:nth-child(3) {
            background: #f5f5f5;
        }
        .list-container li:nth-child(even) {
            background: #e8f4f8;
        }

        /* Not selector */
        .list-container li:not(.special) {
            padding: 5px;
        }
    </style>
</head>
<body>
    <h2>Pseudo-Classes Demo</h2>

    <p><a href="#">Hover over this link</a></p>
    <p><a href="#">Click and hold this link</a></p>

    <p>
        <label for="name">Name:</label>
        <input type="text" id="name" placeholder="Click here to focus">
    </p>

    <h3>Styled List</h3>
    <ul class="list-container">
        <li>First item (bold red)</li>
        <li>Second item (even background)</li>
        <li class="special">Third item (special class)</li>
        <li>Fourth item (even background)</li>
        <li>Fifth item (not special)</li>
        <li>Last item (bold blue)</li>
```

```
        </ul>
    </body>
</html>
```

Part 6: CSS Pseudo-Elements

Introduction to Pseudo-Elements

Pseudo-elements are used to style specific parts of an element or to insert content before or after an element. They are written with two colons (e.g., `::before`, `::after`) or a single colon for older syntax.

Common Pseudo-Elements

Pseudo-element	Description
`::before`	Inserts content before the element's content
`::after`	Inserts content after the element's content
`::first-line`	Styles the first line of a block of text
`::first-letter`	Styles the first letter of a block of text
`::selection`	Styles the portion of text selected by the user
`::placeholder`	Styles the placeholder text in input fields

The ::before and ::after Pseudo-Elements

These pseudo-elements are used to insert generated content. They are commonly used for decorative purposes, icons, and clearfix.

```css
.quote::before {
    content: "\"";
    font-size: 30px;
    color: #3498db;
}
.quote::after {
    content: "\"";
    font-size: 30px;
    color: #3498db;
}
```

Example:

```html
<!DOCTYPE html>
<html>
    <head>
        <title>Pseudo-Elements</title>
        <style>
            .quote::before {
                content: "\201C";
                font-size: 40px;
                color: #3498db;
                margin-right: 5px;
            }
            .quote::after {
                content: "\201D";
                font-size: 40px;
                color: #3498db;
                margin-left: 5px;
            }
            .note::before {
                content: "\2605";
                color: #f39c12;
                margin-right: 10px;
            }
            .note::after {
                content: " (Important)";
                color: #e74c3c;
```

```
            font-size: 12px;
        }

        /* First letter styling */
        .dropcap::first-letter {
            font-size: 50px;
            font-weight: bold;
            color: #e74c3c;
            float: left;
            margin-right: 8px;
        }

        /* First line styling */
        .intro::first-line {
            font-weight: bold;
            color: #2c3e50;
        }

        /* Selection styling */
        ::selection {
            background: #f39c12;
            color: white;
        }
    </style>
</head>
<body>
    <h2>Pseudo-Elements Demo</h2>

    <p class="quote">This is a quotation with decorative quotes using ::before an
d ::after.</p>

    <p class="note">This is a note with a star symbol and additional text using :
:before and ::after.</p>

    <p class="dropcap">This paragraph has a large first letter using ::first-lett
er. The first letter is styled differently to create a drop cap effect commonly used
in magazines and books.</p>
```

```
        <p class="intro">This is an introductory paragraph. The first line is styled differently using ::first-line. Only the first line is bolded.</p>

        <p>Select any text on this page to see the ::selection effect.</p>
    </body>
</html>
```

Part 7: Combining Techniques – Complete Web Page Example

```
html
<!DOCTYPE html>
<html>
    <head>
        <title>Advanced CSS Layout</title>
        <style>
            /* Global Styles */
            * {
                box-sizing: border-box;
                margin: 0;
                padding: 0;
            }
            body {
                font-family: Arial, sans-serif;
                background: #f5f5f5;
            }

            /* Fixed Navigation */
            .navbar {
                position: fixed;
                top: 0;
                left: 0;
                width: 100%;
                background: #2c3e50;
                color: white;
```

```
    padding: 15px 30px;
    z-index: 1000;
    box-shadow: 0 2px 5px rgba(0,0,0,0.3);
}
.navbar a {
    color: white;
    text-decoration: none;
    margin: 0 15px;
    padding: 5px 10px;
}
.navbar a:hover {
    background: #3498db;
    border-radius: 3px;
}

/* Page Content */
.content {
    margin-top: 80px;
    padding: 20px;
    max-width: 1200px;
    margin-left: auto;
    margin-right: auto;
}

/* Hero Section with Gradient */
.hero {
    background: linear-gradient(135deg, #667eea, #764ba2);
    color: white;
    padding: 60px 20px;
    text-align: center;
    border-radius: 10px;
    margin-bottom: 30px;
    position: relative;
}
.hero h1 {
    font-size: 48px;
}
.hero p {
    font-size: 18px;
```

```css
}

/* Image Gallery */
.gallery {
    display: flex;
    flex-wrap: wrap;
    gap: 20px;
    justify-content: center;
    margin: 30px 0;
}
.gallery-item {
    position: relative;
    width: 250px;
    overflow: hidden;
    border-radius: 10px;
    box-shadow: 0 4px 8px rgba(0,0,0,0.2);
    transition: transform 0.3s;
}
.gallery-item:hover {
    transform: scale(1.05);
    z-index: 10;
}
.gallery-item img {
    width: 100%;
    height: 200px;
    object-fit: cover;
    display: block;
}
.gallery-item .overlay {
    position: absolute;
    bottom: 0;
    left: 0;
    right: 0;
    background: rgba(0,0,0,0.7);
    color: white;
    padding: 15px;
    transform: translateY(100%);
    transition: transform 0.3s;
}
```

```
            .gallery-item:hover .overlay {
                transform: translateY(0);
            }

            /* Sticky Footer */
            .footer {
                background: #2c3e50;
                color: white;
                text-align: center;
                padding: 20px;
                margin-top: 30px;
            }
        </style>
    </head>
    <body>
        <nav class="navbar">
            <a href="#">Home</a>
            <a href="#">Gallery</a>
            <a href="#">About</a>
            <a href="#">Contact</a>
        </nav>

        <div class="content">
            <div class="hero">
                <h1>Welcome to My Website</h1>
                <p>This page demonstrates advanced CSS positioning, layout, and desig
n techniques.</p>
            </div>

            <h2>Image Gallery</h2>
            <div class="gallery">
                <div class="gallery-item">
                    <img src="image1.jpg" alt="Nature">
                    <div class="overlay"><h4>Nature</h4><p>Beautiful landscape</p></d
iv>
                </div>
                <div class="gallery-item">
                    <img src="image2.jpg" alt="City">
                    <div class="overlay"><h4>City</h4><p>Urban skyline</p></div>
```

```
                </div>
                <div class="gallery-item">
                    <img src="image3.jpg" alt="Sunset">
                    <div class="overlay"><h4>Sunset</h4><p>Golden hour</p></div>
                </div>
                <div class="gallery-item">
                    <img src="image4.jpg" alt="Forest">
                    <div class="overlay"><h4>Forest</h4><p>Tranquil woods</p></div>
                </div>
            </div>
        </div>

        <footer class="footer">
            <p>&copy; 2024 My Website. All rights reserved.</p>
        </footer>
    </body>
</html>
```

Key Terms

Term	Definition
Position	CSS property that controls how an element is placed in the document
Static	Default positioning; element flows normally in the document
Relative	Positioned relative to its normal position
Absolute	Positioned relative to the nearest positioned ancestor
Fixed	Positioned relative to the viewport; stays fixed on scroll
Sticky	Toggles between relative and fixed based on scroll position
Float	CSS property that moves an element to the left or right, allowing text to wrap
Clear	CSS property that prevents elements from wrapping around floats
Z-index	CSS property that controls the stacking order of positioned elements
Pseudo-class	A keyword added to a selector that specifies a special state of an element
Pseudo-element	A keyword used to style specific parts of an element or insert content
Image Gallery	A collection of images displayed in a grid or organized layout

---

Review Questions

Write your answers on a separate sheet of paper.

1. What is the difference between `position: relative` and `position: absolute`? Give an example of when you would use each.

2. Write a CSS rule to fix a navigation bar to the top of the viewport.

3. What is the purpose of the `float` property in CSS? Give an example of when you would use it.

4. What is the `clear` property and why is it used?

5. What does `z-index` do and what must be true for it to work?

6. Write the HTML and CSS to create an image gallery with at least 4 images arranged in a grid using flexbox.

7. What is the difference between `:hover` and `:active` pseudo-classes?

8. Write a CSS rule that changes the background color of every even-numbered list item.

9. What is the purpose of the `::before` pseudo-element? Give an example.

10. Create a complete HTML page with CSS that demonstrates all of the following:

- A fixed navigation bar
- An image gallery with at least 4 images (use placeholder images)
- Pseudo-classes on links (hover, active, visited)
- Pseudo-elements (::before or ::after)
- A footer at the bottom of the page

Answer Key

1. `position: relative` positions an element relative to its normal position; it still occupies space in the document flow. `position: absolute` removes the element from normal flow and positions it relative to the nearest positioned ancestor. Example: Use `relative` for a container that will contain absolutely positioned child elements. Use `absolute` to place an element precisely within a container.

2.

```
css
nav {
    position: fixed;
    top: 0;
    left: 0;
    width: 100%;
    background: #2c3e50;
    z-index: 1000;
}
```

3. The `float` property moves an element to the left or right, allowing text and other elements to wrap around it. Example: Floating an image inside a paragraph so text wraps around it.

4. The `clear` property specifies which side of a floated element an element should not be next to. It is used to prevent elements from wrapping around floats. Example: `clear: both` forces an element below all floated elements.

5. `z-index` controls the stacking order of positioned elements. A higher value appears in front. For `z-index` to work, the element must be positioned (`position: relative`, `absolute`, `fixed`, or `sticky`).

6.

```
html
<div class="gallery">
    <div class="gallery-item"><img src="img1.jpg" alt="Image 1"></div>
    <div class="gallery-item"><img src="img2.jpg" alt="Image 2"></div>
    <div class="gallery-item"><img src="img3.jpg" alt="Image 3"></div>
```

```
    <div class="gallery-item"><img src="img4.jpg" alt="Image 4"></div>
</div>
```

```
css
.gallery {
    display: flex;
    flex-wrap: wrap;
    gap: 20px;
    justify-content: center;
}
.gallery-item {
    width: 200px;
}
.gallery-item img {
    width: 100%;
    height: auto;
}
```

7. `:hover` applies when the mouse hovers over an element. `:active` applies when the element is being clicked (during the click event).

8.

```
css
li:nth-child(even) {
    background-color: #f0f0f0;
}
```

9. The `::before` pseudo-element inserts content before the element's content. It is commonly used for decorative elements, icons, or adding text.

```
css
.note::before {
    content: "\2605";
    color: gold;
    margin-right: 5px;
}
```

10. Complete HTML page:

```
html
<!DOCTYPE html>
```

```html
<html>
    <head>
        <title>CSS Positioning & Design</title>
        <style>
            /* Reset and base styles */
            * { box-sizing: border-box; margin: 0; padding: 0; }
            body { font-family: Arial, sans-serif; background: #f5f5f5; }

            /* Fixed Navigation */
            .navbar {
                position: fixed;
                top: 0;
                left: 0;
                width: 100%;
                background: #2c3e50;
                color: white;
                padding: 15px 30px;
                z-index: 1000;
            }
            .navbar a {
                color: white;
                text-decoration: none;
                margin: 0 15px;
                padding: 5px 10px;
            }
            .navbar a:hover {
                background: #3498db;
                border-radius: 3px;
            }
            .navbar a:active {
                background: #2ecc71;
            }
            .navbar a:visited {
                color: #bdc3c7;
            }

            /* Page content */
            .content {
                margin-top: 80px;
```

```
    padding: 20px;
    max-width: 1000px;
    margin-left: auto;
    margin-right: auto;
}

/* Gallery */
.gallery {
    display: flex;
    flex-wrap: wrap;
    gap: 20px;
    justify-content: center;
    margin: 30px 0;
}
.gallery-item {
    width: 200px;
    border-radius: 10px;
    overflow: hidden;
    box-shadow: 0 4px 8px rgba(0,0,0,0.2);
    transition: transform 0.3s;
}
.gallery-item:hover {
    transform: scale(1.05);
}
.gallery-item img {
    width: 100%;
    height: 150px;
    object-fit: cover;
    display: block;
}

/* Pseudo-elements */
.quote::before {
    content: "\201C";
    font-size: 30px;
    color: #3498db;
}
.quote::after {
    content: "\201D";
```

```
                font-size: 30px;
                color: #3498db;
            }

            /* Footer */
            .footer {
                background: #2c3e50;
                color: white;
                text-align: center;
                padding: 20px;
                margin-top: 30px;
            }
        </style>
    </head>
    <body>
        <nav class="navbar">
            <a href="#">Home</a>
            <a href="#">Gallery</a>
            <a href="#">About</a>
            <a href="#">Contact</a>
        </nav>

        <div class="content">
            <h1>CSS Positioning & Design</h1>
            <p class="quote">This is a quotation with decorative quotes using ::befor
e and ::after.</p>

            <h2>Image Gallery</h2>
            <div class="gallery">
                <div class="gallery-item"><img src="https://via.placeholder.com/200x1
50" alt="Placeholder"></div>
                <div class="gallery-item"><img src="https://via.placeholder.com/200x1
50" alt="Placeholder"></div>
                <div class="gallery-item"><img src="https://via.placeholder.com/200x1
50" alt="Placeholder"></div>
                <div class="gallery-item"><img src="https://via.placeholder.com/200x1
50" alt="Placeholder"></div>
            </div>
        </div>
```

```
        <footer class="footer">
            <p>&copy; 2024 My Website. All rights reserved.</p>
        </footer>
    </body>
</html>
```